LIFEQUEST · INTERVIEW GUIDE

What this app is, why it exists, and how it is built

A readable brief you can use before a technical interview. It explains the product, the architecture, every library and why it is there, and short answers you can say out loud.

Separate from the project documentation PDF.
App name in the product: LifeQuest. Repository folder: LifeFeast.

1. How to use this in an interview

Start with the product, then the architecture, then one feature you know well. Do not open with a list of library names. A strong opening sounds like this:

> "LifeQuest turns daily habits and tasks into a small quest. You train habits, finish tasks, keep a streak, earn XP, and talk to friends. Friends can share one daily win in the Hall. It is a React single-page app talking to an Express API, with MongoDB for data and JWT for login. It also installs on a phone as a progressive web app."

If they ask "why this stack?", answer with a constraint, not a slogan: one language (JavaScript) on both sides, a document database that matches the nested habit and journal shapes, and a web app that works in the phone browser without an app-store release.

2. What the app is

LifeQuest is a gamified habit tracker. A person signs up, builds daily habits, checks them off, writes a journal, and completes tasks. The app turns that work into levels, XP, coins, streaks, badges, and daily missions. Friends can chat, send challenges, and sit in the Hall to share one daily win and cheer each other.

The people who use it:

- **Player** — tracks habits, tasks, journal, friends, and chat.
- **Admin** — sees users, reports, and private feedback, and can hide a public review.

The live site is a website first. On Android it can be installed from the browser. On iPhone the person uses Safari, then Share, then Add to Home Screen. There is no separate App Store binary.

3. Why it was built this way

- **Habits are easy to skip.** XP, streaks, and a siren for unfinished work make "not done yet" visible.
- **Notes and reminders get lost.** Journal, tasks, and signals live in one account.
- **Doing it alone is harder.** Friends, chat, and challenges add another person.
- **A native app was not required.** A PWA reaches phones from one codebase.
- **English is the default, other languages are optional.** The interface can switch without a new build.

4. What a person can do

Area	What it does
Account	Register, log in, reset password, change theme and language, export data, delete the account.
Training	Create habits by category and difficulty. Mark today done. See a progress ring.
Dashboard	Greeting, level, XP, streak, coins, today's progress, a short quote.
Quest log	Daily, weekly, and monthly tasks. Status is to do, in progress, or completed. Optional notes and a due time.
Journal	One mood and note per calendar day. Not a weekly or monthly notebook.
Missions	Bonus challenges for extra gold. Friend challenges too.
Heroes	Leaderboard plus friend search, requests, accept, decline, and unfriend.
Chat	Messages only between accepted friends. A challenge can be sent from the thread.
Hall	Friends-only daily win wall. Cheer / keep going / you got this. Animated camp companions unlock after a 7-day streak and cost coins.
Celebrations	A short cheer sound and burst when a habit, task, mission, or challenge is completed. Optional in Settings.
Signals	In-app notifications: reminders, friend events, chat messages.
Feedback	Public reviews, or a private note that only the admin sees.
Alarms	A siren and a notification for open habits and to-do tasks that are due. In-progress tasks stay quiet until the next day.

5. How a request moves through the system

1. The person uses the React app in the browser (Vite in development, static files in production).
2. Axios calls `/api/...`. In development, Vite proxies that to the API on port 5001. In production the same origin serves both the site and the API.
3. Express matches the route. Protected routes run the `protect` middleware.
4. That middleware reads the `Authorization: Bearer` JWT, checks it with `JWT_SECRET`, and loads the user from MongoDB without the password.
5. The controller reads or writes Mongoose models and returns JSON.
6. React context (auth, theme, language, unread counts) updates the screen.

> **Health check.** `GET /api/health` answers even if the database is down. A healthy deploy looks like `{"status":"ok","db":"up"}`. If the database is missing, the API returns 503 with a clear message instead of crashing the whole process.

6. Architecture in one picture

Client — React pages, shared layout, contexts, Axios API module, service worker for install and push.

Server — Express app, route files, controllers, Mongoose models, JWT middleware.

Data — MongoDB. Locally this can be a normal MongoDB, or an embedded database if nothing is on port 27017. Production uses MongoDB Atlas. The embedded database is not used on Vercel.

Hosting shape. Vercel runs two services from one project: the Vite site and the Express API. Rewrites send `/api` to the API and everything else to the website. The Express app is the default export. It does not call `listen` on Vercel.

7. Libraries and what each one is for

These are the packages in the app. “Where” means the part of the project that depends on it.

Frontend

Library	Purpose	Why this one
React and React DOM	Build the screens as components and render them in the browser.	The UI is interactive: forms, lists, and context. Components keep each page small.
React Router	URLs such as `/dashboard`, `/habits`, and `/campfire`. Guards send guests to login.	The app is a single page. The router swaps pages without a full reload.
Axios	HTTP client for every API call. Attaches the saved token. On 401 it clears the session.	One place for headers, base URL, and error messages.
Framer Motion	Entrance motion, the Quest-done overlay (AnimatePresence), and companion idle / cheer motion. Also used on dashboard cards and toasts.	Motion is a detail, not the product. We did not add Lottie, Three.js, or a game engine for the camp characters.
Lucide React	Icons in the sidebar and buttons (menu, bell, settings, coins, flame for Hall).	Consistent line icons instead of a mix of image files.
Recharts	Charts on the Chronicle / analytics page.	Progress over days and weeks is easier to see as a chart than as a table.
Vite and the React plugin	Development server, fast refresh, and the production build.	The client is plain React, not Next.js. Vite only builds the browser app.

Backend

Library	Purpose	Why this one
Express 5	HTTP server and routes under `/api`.	A small REST API. Controllers stay separate from the route list.
Mongoose	Models, validation, and queries for users, habits, tasks, journal, messages, Hall wins, and the rest.	Schemas document the shape of each collection and reject bad writes.
MongoDB Memory Server	Starts a temporary MongoDB on a laptop when no database is running.	A new developer can run the API without installing MongoDB. Production sets `MONGODB_URI` to Atlas instead.
jsonwebtoken	Creates and checks login tokens.	The API is stateless. The browser stores the token and sends it on later calls.
bcryptjs	Hashes passwords before save and compares them on login. Cost factor 12.	Passwords are never stored in plain text. A pure-JS hash avoids native build issues.
cors	Allows the Vite dev origin and the deployed site to call the API.	Browsers block cross-origin calls unless the API says yes. Same-origin production still lists known hosts.
dotenv	Loads `server/.env` locally.	Secrets stay out of source code. On Vercel the same names are project environment variables.
web-push	Sends Web Push messages to browsers that subscribed with a VAPID key.	Reminders and chat alerts can reach a closed tab when push is allowed. If keys are missing, push is skipped.

Tools around the app, not inside a request

Library	Purpose
concurrently	Runs the API and the Vite app with one `npm run dev`.
nodemon	Restarts the API when server files change.
oxlint	Lints the client. It does not ship to users.

The database itself is MongoDB (Atlas in production). It is not an npm package. Mongoose is the library that talks to it.

8. Data we store

Model	Holds
User	Name, email, password hash, role, XP, level, coins, streaks, badges, friends, companions owned, equipped companion, theme, language.
Habit	Name, category, difficulty, reminder, and which dates were completed.
Task	Daily, weekly, or monthly. Status, due time, notes, and when it was marked in progress.
Journal	One entry per user per date: mood, text, tags.
Mission / Challenge	Daily missions, and challenges sent between friends.
FriendRequest	Pending, accepted, or declined.
Message	Chat text from one friend to another.
Achievement	One Hall win per user per day: short text, habit snapshot, companion, reactions.
Notification	Signals: reminders, friend events, chat previews. Read or unread.
Feedback	Public review or private improvement note. Admin can hide a review.
PushSubscription	Browser push endpoint and keys for one device.

Level uses a curve: XP needed for the next level is about `100 × (level ^ 1.5)`. A level-up also adds 50 coins. Habit XP is 10, 20, or 40 for easy, medium, and hard.

9. Login, in plain steps

1. Register sends name, email, and password. The password must be at least 6 characters. Mongoose hashes it on save.
2. Login checks the email and `bcrypt.compare`. Inactive accounts are rejected.
3. The API returns a JWT and a user object with no password.
4. The client saves the token and user in `localStorage` and sends the token on later requests.
5. A protected page waits until `/api/auth/me` finishes. If there is no user, the router sends them to login.

Forgot-password creates a reset token. In this project the token is returned in the API response so a local demo can finish the reset without email infrastructure. In a production interview answer, say you would email the token and never return it in JSON.

10. Features worth explaining out loud

In-progress tasks and the siren

A task is `todo`, `in-progress`, or `completed`. The alarm ignores completed tasks. If you mark a task in progress, the siren stays quiet for the rest of that local day and can return the next day if it is still not done. The server stores `inProgressAt` when the status changes. Habits are separate: they alarm until they are completed today.

Friends and chat

You search Heroes by name or email, send a request, and chat only after it is accepted. Unfriend removes both sides of the friend list, closes open challenges, and notifies the other person. Chat is not a live socket. The open thread refreshes every few seconds. A new message also creates a Signal and, if push is set up, a Web Push. While the app is open, a poll shows a toast if you are not already looking at that thread.

Hall, camp UI, and celebrations – libraries used (interview talking point)

These features did **not** add a new npm package. They reuse the stack already in the project. That is a good interview line: we chose not to pull in Three.js, Lottie, Socket.IO, or a social SDK.

What you see	Library or browser API	What we wrote ourselves
Hall page, feed, shop	React, React Router (`/hall`), Axios (`hallAPI`)	`Hall.jsx`, `/api/hall`, `Achievement` model
Cheer / keep going / you got this	React state + Express + Mongoose	One reaction per user per post; Signal with `meta.kind = hall-cheer`
Living camp flame and lantern cards	CSS (keyframes, gradients). No extra CSS library.	`CampFlame.jsx` and camp rules in `global.css`
Stained-glass companions	Inline SVG in React. Framer Motion is not required for the drawing.	`CampCompanion.jsx` — Ember, Fox, Knight, Owl
Companion idle / wave	CSS animation + a Framer Motion overlay on complete	Mood class `is-cheer`; reduced-motion media query
Quest-done burst	Framer Motion `AnimatePresence`	`CelebrateBurst.jsx`, `CelebrateContext`
Cheer sound	Web Audio API (browser). No Howler, no MP3 file.	`cheer.js` — short triangle fanfare on the same AudioContext as the siren
Hall icon	Lucide React `Flame`	Nav entry in `AppLayout.jsx`
Unlocks	Mongoose user fields + coins already on the user	7-day streak (current or longest) opens the shop; 80 / 120 / 200 coins buy extras

Routes: `GET/POST /api/hall`, `POST /api/hall/:id/react`, `GET /api/hall/shop`, `POST /api/hall/shop/buy`, `POST /api/hall/shop/equip`. Feed is friends plus you. One post per calendar day.

If they ask “why not a new library?”, say: a camp flame and four SVG silhouettes are cheaper and more unique than a 3D kit that looks like every other app. The sound is generated so there is no audio asset to host. Chat stays polling; Hall is request/response because a daily win does not need a socket.

Journal versus task notes

The journal is daily only. Weekly and monthly “notes” are an optional text field on a weekly or monthly task. Nothing auto-writes a weekly summary.

Languages

English is the default. Hindi, Spanish, French, German, Portuguese, Arabic, Chinese, and Japanese are merged on top of English, so a missing phrase falls back to English. Arabic sets the page

direction to right-to-left. The choice is saved on the user and in local storage.

Phones

The layout uses a real viewport, safe-area padding, and dynamic viewport height so the iPhone browser bar does not cover the bottom. Inputs are 16px so Safari does not zoom on focus. Login fields have autocomplete. The manifest does not lock portrait, so iPad and Mac can rotate. Push on iPhone works best after Add to Home Screen. If `PushManager` is missing, subscribe is skipped.

11. Security points you should be able to say

- Passwords are hashed with bcrypt. The API never sends the hash back.
- JWT is signed with a server secret. The middleware rejects a bad or expired token.
- Admin routes check `role === 'admin'` after login.
- Chat, unfriend, and Hall reactions check that the two users are actually friends. A Hall post is one short string per day, not an open comment thread.
- Feedback privacy: public reviews are on the wall; improvement notes are admin-only.
- Secrets live in environment variables, not in the React bundle. `VITE_` variables are public. `JWT_SECRET` and `MONGODB_URI` are server-only.
- Do not put a GitHub token inside `git remote`. Use SSH or a prompt.

12. Deploy, short version

Production needs `MONGODB_URI` (Atlas, network access open to the host), `JWT_SECRET`, and `CLIENT_URL`. Leave the client API URL unset so the site calls same-origin `/api`. On Vercel the API must not call `process.exit` if the database is slow, and it must not register an Express 5 route of `app.get('*')`. That pattern throws while the module loads and the platform reports a function crash. The SPA is served by the web service. Express only serves the built files when it is a single process, such as Render or `npm start`.

13. Interview questions and answers

Say these in your own words. One or two sentences, then offer to go deeper.

About the project

What did you build?

A gamified habit tracker called LifeQuest. Users complete habits and tasks, earn XP and streaks, journal, chat with friends, and share one daily win in the Hall. It is a React PWA with an Express and MongoDB API.

Why not a native iOS and Android app?

One web codebase already runs on phones. Install is “add to home screen”. Push is limited on iPhone until it is installed, and the UI says that instead of pretending there is an App Store build.

What was a hard bug?

On Vercel the API returned a function crash before login could run. Two causes: connecting to MongoDB at import time and exiting the process, and an Express 5 catch-all route `app.get('*')` that throws as soon as the file loads. The fix was a lazy database connect that returns 503, and a route that Express 5 accepts, used only when Express itself serves the website.

How do phones avoid a blank screen after a deploy?

Hashed JavaScript files change every build. If a service worker returns the old HTML, or returns HTML for a missing script, the page stays blank. The worker prefers a fresh HTML document, does not substitute HTML for `/assets`, and the server never sends `index.html` for a missing asset.

Frontend

Why React context instead of Redux?

The shared state is small: the logged-in user, theme, language, and unread counts. Context is enough. I would add a store only if many distant screens started writing the same fast-changing state.

How does routing protect pages?

A wrapper reads the auth context. While the token is being checked it shows a loader. If there is no user it navigates to login and remembers the path. Admin pages also require the admin role.

Why Axios?

Every call needs the same base URL and bearer token. Interceptors do that once. Fetch would work, but I would still wrap it.

Is chat real-time?

No WebSocket. The open chat polls every few seconds. New messages also become notifications, and push is sent if the friend subscribed. Polling is simpler to host. Sockets would be the next step if delay mattered.

What libraries did you add for the Hall and the camp look?

None. Hall uses React, React Router, Axios, Express, and Mongoose — already in the project. The flame and lanterns are CSS. Companions are inline SVG. The cheer sound is the Web Audio API. Framer Motion already handled overlays, so the Quest-done card uses that. We did not add Three.js, Lottie, Howler, or Socket.IO.

How do companions unlock?

Everyone gets Ember. A 7-day streak, current or longest, opens the shop. Extra companions cost 80, 120, or 200 coins already stored on the user. The API rejects a buy if the week gate is closed or coins are short.

How does language switching work?

Each language is an object of strings. Non-English packs are deep-merged over English. A function looks up a dotted key and fills `{{name}}` placeholders. The document `lang` and `dir` update with the choice.

Backend and data

Why MongoDB?

Habits store a list of completion dates, users store badges and friend ids, and notifications store a flexible meta object. Documents match that better than a large set of join tables for this size of app.

Why Mongoose if MongoDB is schemaless?

The database is flexible. The app is not. Mongoose enums stop a bad status, required fields stop empty users, and hooks hash the password in one place.

How is the password stored?

A pre-save hook hashes it with bcrypt, 12 rounds, only when the password changed. Login uses compare. The hash is excluded when the user is loaded for a request.

Why JWT instead of server sessions?

The API can run as a serverless function without a shared session store. The token carries the user id. The downside is logout is local: we delete the token in the browser. A stolen token works until it expires, so the secret must stay on the server and the expiry is limited.

How do you stop one user from reading another user's tasks?

Every query includes the id from the verified token, for example `Task.find({ user: req.user._id })`. The client does not choose the owner id.

What happens if Atlas is down on Vercel?

The function still boots. Health returns 503 with `db: down`. Login returns the same class of error instead of a platform crash. The connection is cached so warm invocations reuse it.

Product rules

When does the siren run?

For habits not done today, and for tasks that are still to do and whose due time has passed. In-progress tasks wait until the next local day. Completed tasks never alarm. The sound also needs the browser to have been unlocked by a tap, because autoplay rules block audio otherwise.

Can I unfriend someone?

Yes, from Heroes and from the chat header. Both friend lists update, open challenges close, and the other person gets a signal.

Are weekly notes a separate journal?

No. The journal is one entry per day. Weekly and monthly are task lists. A task can have a notes field when you create it.

If they push on tradeoffs

What would you change next?

Email the password-reset token instead of returning it. Add a short-lived refresh token or a denylist if logout must kill a stolen JWT. Move chat to WebSockets if polling load grows. Add indexes on user id and created time for messages and notifications.

What would you not claim?

I would not call the chat realtime, the PWA a native app, or the embedded MongoDB a production database. I would not say the client can hide secrets.

14. A 60-second walkthrough

1. Open with the sentence in section 1.
2. Draw three boxes: React, Express, MongoDB. Arrow from browser to `/api` with a JWT.
3. Pick one feature. Habits are the easiest: create, complete today, XP and streak update, dashboard reads the summary.
4. Mention one constraint you handled: phones (safe area, install) or deploy (database must not crash the function).
5. Stop and ask what they want next: auth, data model, or a bug.

This guide describes the LifeQuest codebase as built in this repository. It is for explanation. It is not a second copy of the user-facing app.